Annotated log of conversation with Mistral AI's Le Chat interface for solving HW3 coding

Prompt 1:

PDF q_mup_coding.pdf

I need help with part b): "Examing the norms of the updates to the weights". Specifically, help me write code for those three norms I indicated.

Le Chat's answer:

Here's how you can compute the **Frobenius norm**, **spectral norm**, and **RMS-RMS induced norm** for the weight updates in the provided code.

What You Need to Do

For each weight matrix update (`delta_params`), compute:

1. **Frobenius norm**
2. **Spectral norm**
3. **RMS-RMS induced norm** (this is the Frobenius norm of the update, normalized by the square root of the input dimension)

Code Implementation for Part b

Copy

```
batch_ids = np.random.randint(0, len(train_images), size=64)

def train_one_step_matrices(mlp=MLP, hiddens=[0, 16, 64, 64, 64, 256, 256, 1024], optimizer
    model = mlp(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)

    old_params = [p.detach().clone() for p in model.parameters()]

    for i in range(1):
        images_batch = train_images[batch_ids]
        labels_batch = train_labels[batch_ids]
        images_batch, labels_batch = images_batch.to(device), labels_batch.to(device)

        optimizer.zero_grad()
        outputs, activations = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

    new_params = [p.detach().clone() for p in model.parameters()]
    delta_params = [new_p - old_p for new_p, old_p in zip(new_params, old_params)]
```

```
    frob_norms = []
    spectral_norms = []
    induced_norms = []
    p_shapes = []

    for p in delta_params:
        if len(p.shape) == 2:  # Only compute for weight matrices, not biases
            # Frobenius norm
            frob_norm = torch.linalg.norm(p, ord='fro')
            frob_norms.append(frob_norm.item())

            # Spectral norm
            spectral_norm = torch.linalg.norm(p, ord=2)
            spectral_norms.append(spectral_norm.item())

            # RMS-RMS induced norm (Frobenius norm / sqrt(input_dim))
            input_dim = p.shape[1]
            induced_norm = frob_norm / np.sqrt(input_dim)
            induced_norms.append(induced_norm.item())

            p_shapes.append(f"{p.shape[0]}x{p.shape[1]}")

    fig, axs = plt.subplots(1, 3, figsize=(20, 6))
```

```
    axs[0].set_title(f"Frobenius norm of update per layer ({label})")
    axs[0].set_xlabel("Hidden Size of layer")
    axs[0].bar(np.arange(len(frob_norms)), frob_norms)
    axs[0].set_xticks(np.arange(len(frob_norms)))
    axs[0].set_xticklabels(p_shapes, rotation=45, ha='right')

    axs[1].set_title(f"Spectral norm of update per layer ({label})")
    axs[1].set_xlabel("Hidden Size of layer")
    axs[1].bar(np.arange(len(spectral_norms)), spectral_norms)
    axs[1].set_xticks(np.arange(len(spectral_norms)))
    axs[1].set_xticklabels(p_shapes, rotation=45, ha='right')

    axs[2].set_title(f"Induced norm of update per layer ({label})")
    axs[2].set_xlabel("Hidden Size of layer")
    axs[2].bar(np.arange(len(induced_norms)), induced_norms)
    axs[2].set_xticks(np.arange(len(induced_norms)))
    axs[2].set_xticklabels(p_shapes, rotation=45, ha='right')

    plt.show()

train_one_step_matrices(optimizer=SimpleAdam)
```

Explanation of the Code

- **Frobenius norm**: Computed using `torch.linalg.norm(p, ord='fro')`.
- **Spectral norm**: Computed using `torch.linalg.norm(p, ord=2)`.
- **RMS-RMS induced norm**: Computed as the Frobenius norm divided by the square root of the input dimension (`p.shape[1]`).

Expected Output

- Three bar plots showing the Frobenius norm, spectral norm, and RMS-RMS induced norm for each layer's weight update.
- The x-axis labels show the shape of each weight matrix.

Which Norm Correlates Most with RMS of Activations?

- The **RMS-RMS induced norm** (Frobenius norm normalized by input dimension) should correlate most closely with the RMS of activation deltas, as it accounts for the scaling effect of the input dimension.

Comment 1: Le Chat was able to arrive at the correct answer one shot.

Prompt 2.1:

Now help me with part c: Implementing muP

Le Chat's answer:

Let's implement **Maximal Update Parameterization (µP)** in the `SimpleAdamMuP` optimizer. The goal is to adjust the per-layer learning rate so that the RMS of activation deltas is uniform across layers, regardless of their size.

Key Concepts for µP

- **µP scaling**: For a layer with input dimension d_{in} and output dimension d_{out}, the learning rate should be scaled by $\frac{1}{\sqrt{d_{\text{in}}}}$.
- **Special cases**:
  - For biases, use a small fudge factor (e.g., 0.01).
  - For the input layer, use a fudge factor of 16.
  - For the output layer, ignore µP scaling and use a fixed learning rate (e.g., 0.003).

Implementation of `SimpleAdamMuP`

Copy

```
from torch.optim.optimizer import Optimizer
from typing import Any

class SimpleAdamMuP(Optimizer):
    def __init__(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2)
        super(SimpleAdamMuP, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for i, p in enumerate(group["params"]):
                grad = p.grad.data
                state = self.state[p]
                if len(state) == 0:  # Initialization
                    state["step"] = torch.tensor(0.0)
```

```
            state["step"] += 1
            m = state["momentum"]
            m.lerp_(grad, 1 - group["b1"])
            v = state["variance"]
            v.lerp_(grad**2, 1 - group["b2"])

            m_hat = m / (1 - group["b1"] ** state["step"])
            v_hat = v / (1 - group["b2"] ** state["step"])
            u = m_hat / (torch.sqrt(v_hat) + 1e-16)

            # Adjust the per-layer learning rate scaling factor
            lr = group["lr"]
            if len(p.shape) == 2:  # Weight matrix
                d_in = p.shape[1]
                # Input layer: fudge factor of 16
                if i == 0:
                    lr = lr * 16 / np.sqrt(d_in)
                # Output layer: ignore µP scaling
                elif i == len(group["params"]) - 1:
                    lr = 0.003
                # Hidden layers: standard µP scaling
                else:
                    lr = lr / np.sqrt(d_in)
            else:  # Bias
                lr = lr * 0.01

            p.add_(u, alpha=-lr)
    return None
```

Explanation of the Code

- **Initialization**: The optimizer initializes momentum and variance buffers for each parameter.
- **Momentum and Variance Update**: The optimizer updates the momentum and variance using exponential moving averages.
- **Learning Rate Scaling**:
  - For **weight matrices**, the learning rate is scaled by $\frac{1}{\sqrt{d_{\text{in}}}}$.
  - For the **input layer**, a fudge factor of 16 is applied.
  - For the **output layer**, µP scaling is ignored, and a fixed learning rate of 0.003 is used.
  - For **biases**, a small fudge factor of 0.01 is applied.

How to Use the Optimizer

Copy

```
train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[0, 16, 64, 64, 64,
train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[0, 16, 32, 64, 128
```

Expected Behavior

- The RMS of activation deltas should be uniform across layers, regardless of their width.
- The first 2-3 layers may have slightly lower RMS deltas due to the fudge factors and the nature of the input data.

Why the First Few Layers May Have Lower Norms

- The input layer and the first few hidden layers often have different statistical properties due to the nature of the input data and the initial random weights.
- The fudge factors (e.g., 16 for the input layer) are empirical adjustments to account for these differences.

Comment 2.1: The code contains error message.

Prompt 2.2:

```
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-1032906217.py in <cell line: 0>()
    60                 p.add_(u, alpha=-lr)
    61         return None
---> 62 train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[8, 16,
64, 64, 64, 256, 256, 1024])
    63 train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[8, 16, 32,
64, 128, 256, 512, 1024])
3 frames
/tmp/ipython-input-1032906217.py in step(self)
    45                 d_in = p.shape[1]
    46                 # Input layer: fudge factor of 16
---> 47                 if i == 0:
    48                     lr = lr * 16 / np.sqrt(d_in)
    49                 # Output layer: ignore µP scaling
NameError: name 'i' is not defined
```

Le Chat's answer:

The error occurs because the variable `i` (the index of the parameter in the parameter group) is not defined in the scope where it is being used. This happens because the loop variable `i` is only available inside the `for` loop, but the code tries to use it outside of that context.

To fix this, we need to ensure that the logic for identifying the input, hidden, and output layers is robust and does not rely on the loop index `i`. Instead, we can use the shape of the parameter to infer its role (input, hidden, or output layer).

Updated Implementation

Here's the corrected version of the `SimpleAdamMuP` optimizer:

Copy

```
from torch.optim.optimizer import Optimizer
import torch
import numpy as np

class SimpleAdamMuP(Optimizer):
    def __init__(self, params, lr=1e-1, b1=0.9, b2=0.999):
        defaults = dict(lr=lr, b1=b1, b2=b2)
        super(SimpleAdamMuP, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            params = list(group["params"])
            for idx, p in enumerate(params):
                if p.grad is None:
                    continue
                grad = p.grad.data
                state = self.state.setdefault(p, {})

                if len(state) == 0:
```

```
                state["step"] += 1
                m = state["momentum"]
                m.lerp_(grad, 1 - group["b1"])
                v = state["variance"]
                v.lerp_(grad**2, 1 - group["b2"])

                m_hat = m / (1 - group["b1"] ** state["step"])
                v_hat = v / (1 - group["b2"] ** state["step"])
                u = m_hat / (torch.sqrt(v_hat) + 1e-16)

                lr = group["lr"]
                if len(p.shape) == 2:  # Weight matrix
                    d_in = p.shape[1]
                    # Check if this is the first layer (input layer)
                    if idx == 0:
                        lr = lr * 16 / np.sqrt(d_in)
                    # Check if this is the last layer (output layer)
                    elif idx == len(params) - 1:
                        lr = 0.003
                    # Hidden layers
                    else:
                        lr = lr / np.sqrt(d_in)
                else:  # Bias
                    lr = lr * 0.01

                p.add_(u, alpha=-lr)
        return None
```

Key Changes

- The loop variable is now named `idx` and is used to identify the first and last layers.
- The logic for identifying input, hidden, and output layers is based on the index of the parameter in the `params` list.

How to Use

```
model = MLP(hidden_sizes=[8, 16, 64, 64, 64, 256, 256, 1024]).to(device)
optimizer = SimpleAdamMuP(model.parameters(), lr=2)
train_one_step(optimizer=optimizer, lr=2, label="Adam MuP", hiddens=[8, 16, 64, 64, 64, 256
```

Comment 2.2: Le Chat seems to deviate from the original implementation of the code and writes its own solution. I have to manually prevent it from deviating too much from the original version of the code.

Prompt 2.3:

> I noticed that your version of the code is different from the doc I shared with you. In the doc I shared with you, there is only "for p in group['params']:" without an additional argument

Le Chat's answer:

You're absolutely right. The original code in the document you shared uses only `for p in group["params"]:` without an explicit index. To handle the input, hidden, and output layers correctly, we need to infer the layer type based on the parameter's shape and its position in the model's parameter list.

However, since we don't have direct access to the model's architecture inside the optimizer, we need to use a workaround. We can use the parameter's shape to infer if it's a weight matrix or bias, and then apply the appropriate scaling.

Here's how you can implement the `SimpleAdamMuP` optimizer while sticking to the original loop structure:

Implementation of `SimpleAdamMuP`

Copy

```
from torch.optim.optimizer import Optimizer
import torch
import numpy as np

class SimpleAdamMuP(Optimizer):
    def __init__(self, params, lr=1e-1, b1=0.9, b2=0.999):
        defaults = dict(lr=lr, b1=b1, b2=b2)
        super(SimpleAdamMuP, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad.data
                state = self.state.setdefault(p, {})

                if len(state) == 0:
                    state["step"] = torch.tensor(0.0)
                    state["momentum"] = torch.zeros_like(p, dtype=torch.float32)
                    state["variance"] = torch.zeros_like(p, dtype=torch.float32)
```

```
            state["step"] += 1
            m = state["momentum"]
            m.lerp_(grad, 1 - group["b1"])
            v = state["variance"]
            v.lerp_(grad**2, 1 - group["b2"])

            m_hat = m / (1 - group["b1"] ** state["step"])
            v_hat = v / (1 - group["b2"] ** state["step"])
            u = m_hat / (torch.sqrt(v_hat) + 1e-16)

            lr = group["lr"]
            if len(p.shape) == 2:  # Weight matrix
                d_in = p.shape[1]
                # Assume first weight matrix is the input layer
                if p is list(group["params"])[0]:
                    lr = lr * 16 / np.sqrt(d_in)
                # Assume last weight matrix is the output layer
                elif p is list(group["params"])[-2]:
                    lr = 0.003
                # Hidden layers
                else:
                    lr = lr / np.sqrt(d_in)
            else:  # Bias
                lr = lr * 0.01

            p.add_(u, alpha=-lr)
        return None
```

Key Points

- **Input Layer**: The first weight matrix in `group["params"]` is assumed to be the input layer.
- **Output Layer**: The second-to-last parameter (since the last parameter is typically a bias) is assumed to be the output layer.
- **Hidden Layers**: All other weight matrices are treated as hidden layers.
- **Bias**: All 1D parameters are treated as biases and scaled by a fudge factor of 0.01.

How to Use

Copy

```
model = MLP(hidden_sizes=[8, 16, 64, 64, 64, 256, 256, 1024]).to(device)
optimizer = SimpleAdamMuP(model.parameters(), lr=2)
train_one_step(optimizer=optimizer, lr=2, label="Adam MuP", hiddens=[8, 16, 64, 64, 64, 256
```

Comment 2.4: After forcing it to stay up with the original version of the code, Le Chat was able to correct its behavior and arrive at the correct solution.

Prompt 3:

Now, I need help with part d: "Pre-Weight Multipliers"

Le Chat's answer:

Let's tackle **Part d: Per-Weight Multipliers** in your assignment. The goal is to implement Maximal Update Parameterization (μP) by adjusting the "network graph" itself, rather than the optimizer. This means you'll scale the output of each layer by a constant, which is equivalent to adjusting the learning rate for that layer when using optimizers like Adam or Sigmoid.

Key Concepts for Per-Weight Multipliers

- **Scaling the Output**: Multiplying the output of a layer by a constant is equivalent to scaling the learning rate for that layer's weights.
- **μP Scaling**: For a layer with input dimension d_{in}, the output should be scaled by $\frac{1}{\sqrt{d_{\text{in}}}}$.
- **Special Cases**:
  - For the input layer, use a fudge factor of 16.
  - For the output layer, do not apply μP scaling.
  - For biases, use a small fudge factor of 0.01.

Implementation of `ScaledMLP`

Here's how you can implement the `ScaledMLP` class with per-weight multipliers:

```
import torch
import torch.nn as nn
import numpy as np

class ScaledMLP(nn.Module):
    def __init__(self, input_size=784, hidden_sizes=[8, 16, 32, 64, 128], num_classes=10):
        super(ScaledMLP, self).__init__()
        all_hidden_sizes = [input_size] + hidden_sizes + [num_classes]
        self.layers = nn.ModuleList()
        self.scalers = nn.ParameterList()

        for i in range(len(all_hidden_sizes) - 1):
            self.layers.append(nn.Linear(all_hidden_sizes[i], all_hidden_sizes[i + 1], bias

            # Add a scaler for each layer except the output layer
            if i < len(all_hidden_sizes) - 2:
                d_in = all_hidden_sizes[i]
                if i == 0:  # Input layer
                    scale = 16 / np.sqrt(d_in)
                else:  # Hidden layers
                    scale = 1 / np.sqrt(d_in)
                scaler = nn.Parameter(torch.tensor(scale, dtype=torch.float32), requires_gr
                self.scalers.append(scaler)
            else:  # Output layer
                self.scalers.append(nn.Parameter(torch.tensor(1.0, dtype=torch.float32), re

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        activations = []
        x = x.view(x.size(0), -1)  # Flatten: (batch_size, 28*28)

        for i, layer in enumerate(self.layers[:-1]):
            x = layer(x)
            x = x * self.scalers[i]  # Apply scaling
            x = self.sigmoid(x)
            activations.append(x)

        x = self.layers[-1](x)  # Output layer, no scaling
        activations = activations[1:]  # Exclude input layer activations

        return x, [a.detach() for a in activations]
```

Explanation of the Code

- **Initialization**:
  - The `ScaledMLP` class initializes the layers and scalers.
  - For each layer, a scaler is added to the `scalers` list, except for the output layer.
  - The scaler for the input layer is set to $\frac{16}{\sqrt{d_{\text{in}}}}$.
  - The scaler for hidden layers is set to $\frac{1}{\sqrt{d_{\text{in}}}}$.
  - The scaler for the output layer is set to 1 (no scaling).
- **Forward Pass**:
  - The output of each layer is scaled by the corresponding scaler before applying the activation function.
  - The output layer is not scaled.

How to Use

Copy

```
model = ScaledMLP(hidden_sizes=[8, 16, 64, 64, 64, 256, 256, 1024]).to(device)
optimizer = SimpleAdam(model.parameters(), lr=2)
train_one_step(mlp=model, optimizer=optimizer, label="Scaled MLP", hiddens=[8, 16, 64, 64,
```

Why Multiplying the Output is Equivalent to Adjusting the Learning Rate

- **Mathematical Equivalence**:
  - If you multiply the output of a layer by a constant c, the gradient during backpropagation will also be scaled by c.
  - This is equivalent to scaling the learning rate for that layer's weights by c when using optimizers like Adam or SGD.
- **Example**:
  - Let $y = c \cdot (Wx)$.
  - The gradient $\frac{\partial L}{\partial W}$ will be scaled by c.
  - This is equivalent to using a learning rate of $c \cdot \eta$ for W.

Comment 3.1: Le Chat's answer deviates from the original implementation again. So I need to manually enforce its behavior again.

Prompt 3.2:

I noticed that you defined another variable called scalers. However, please make all code part the same to the doc I shared with you except the part "To do". This is the error message I got again:

"AttributeError Traceback (most recent call last)
/tmp/ipython-input-3843602231.py in <cell line: 0>()
22 return x, [a.detach() for a in activations]
23
——→ 24 train_one_step(mlp=ScaledMLP, optimizer=SimpleAdam)
4 frames
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in __getattr__(self, name)
1960 if name in modules:
1961 return modules[name]
→ 1962 raise AttributeError(
1963 f"'{type(self).__name__}' object has no attribute '{name}'"
1964)
AttributeError: 'ScaledMLP' object has no attribute 'scalers'"

Le Chat's answer:

```python
class ScaledMLP(nn.Module):
    def __init__(self, input_size=784, hidden_sizes=[8, 16, 32, 64, 128], num_classes=10):
        super(ScaledMLP, self).__init__()
        all_hidden_sizes = [input_size] + hidden_sizes + [num_classes]
        self.layers = nn.ModuleList()
        for i in range(len(all_hidden_sizes) - 1):
            self.layers.append(nn.Linear(all_hidden_sizes[i], all_hidden_sizes[i + 1], bias
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        activations = []
        x = x.view(x.size(0), -1)  # Flatten: (batch_size, 28*28)

        for layer in self.layers[:-1]:
            x = layer(x)
            # Scale the output of each layer by the appropriate μP factor
            d_in = layer.weight.shape[1]
            if layer is self.layers[0]:  # Input layer
                x = x * (16 / np.sqrt(d_in))
            else:  # Hidden layers
                x = x * (1 / np.sqrt(d_in))

            x = self.sigmoid(x)
            activations.append(x)

        x = self.layers[-1](x)  # Output layer, no scaling
        activations = activations[1:]  # Exclude input layer activations

        return x, [a.detach() for a in activations]
```

Comment 3.3: After forcing it to stay up with the original version of the code, Le Chat was able to correct its behavior and arrive at the correct solution.

Prompt 4:

Now let's move on to part e) Hyperparameter Transfer. Help me answer the following two questions:
1)Run the following code, which will perform a sweep over learning rates for 3-layer MLPs of increasing width using Adam. How does the optimal learning rate change as the network increases in size?
2)In the second cell, we will instead use the muP optimizer you implemented. How does the optimal learning rate work now? You should aim to show that there is a single global learning rate that works on a majority of widths. The 256-width network should achieve a loss of 0.5, comparable to Adam.

Le Chat's answer:

Let's address both questions step-by-step, focusing on interpreting the results of the learning rate sweeps for **Adam** and **µP (Maximal Update Parameterization)** optimizers.

---

1. Optimal Learning Rate for Adam as Network Width Increases

What the Code Does

The provided code performs a sweep over learning rates for 3-layer MLPs of increasing width using the **Adam** optimizer. It trains each network for 100 steps and records the validation loss for each learning rate.

Expected Observations

- **Optimal Learning Rate Decreases with Width**:
  As the width of the network increases, the optimal learning rate typically decreases. This is because wider networks have more parameters, and larger updates can lead to instability or divergence.

- **Visualization**:
  The plot of loss vs. learning rate for different widths will show that the optimal learning rate shifts left (to smaller values) as the width increases.

2. Optimal Learning Rate for µP Optimizer

What the Code Does

The second cell performs a similar sweep, but using the **µP optimizer** you implemented. µP aims to make the network's training dynamics invariant to width, allowing a single learning rate to work across different widths.

Expected Observations

- **Single Global Learning Rate**:
  With µP, you should observe that a single learning rate works well across all widths. The optimal learning rate should not change significantly as the width increases.

- **Performance**:
  The 256-width network should achieve a loss of around 0.5, comparable to the performance with Adam.

How to Answer the Question

- **Run the Code**:
  Execute the cell that performs the learning rate sweep for µP. This will generate a plot similar to the Adam plot.

How to Answer the Question

- **Run the Code**:
  Execute the cell that performs the learning rate sweep for Adam. This will generate a plot showing the loss for each learning rate and width.

- **Interpret the Plot**:
  - The x-axis represents the learning rates.
  - The y-axis represents the validation loss.
  - Each curve represents a different network width.

- **Optimal Learning Rate**:
  For each width, identify the learning rate that results in the lowest loss. Note how this optimal learning rate changes as the width increases.

Example Interpretation

Adam Sweep

- **Plot Interpretation**:
  - For width 4, the optimal learning rate might be around 0.01.
  - For width 64, the optimal learning rate might be around 0.003.
  - For width 256, the optimal learning rate might be around 0.001.
- **Conclusion**:
  The optimal learning rate decreases as the network width increases.

µP Sweep

- **Plot Interpretation**:
  - A learning rate of 0.3 might work well for all widths (4, 8, 16, 32, 64, 128, 256).
  - The loss for the 256-width network at this learning rate should be around 0.5.
- **Conclusion**:
  µP allows for a single global learning rate to work across different network widths, demonstrating its effectiveness in hyperparameter transfer.

1. Optimal Learning Rate for Adam

- The optimal learning rate decreases as the network width increases. Wider networks require smaller learning rates to train effectively.

2. Optimal Learning Rate for µP

- With µP, there is a single global learning rate that works well across all network widths. The 256-width network achieves a loss of around 0.5, comparable to the performance with Adam.

How to Present Your Answer

- **For Question 1**:

  > ""As the network width increases, the optimal learning rate for Adam decreases. This is evident from the plot, where wider networks achieve the lowest loss at smaller learning rates.""

- **For Question 2**:

  > ""With the µP optimizer, a single global ↓ rning rate (e.g., 0.3) works well across all

Comment 4: Le Chat was able to arrive at the correct answer one shot.

Prompt 5:

Sorry for the cutoff. Let's now move on to the last part f)(Shampoo and Orthogonalization). Help me implement the simplified Shampoo update.

Le Chat's answer:

Let's implement the **simplified Shampoo update** for the `SimpleShampoo` optimizer. The Shampoo update involves performing a Singular Value Decomposition (SVD) on the momentum matrix and using the resulting orthogonal matrices for the update.

Key Concepts for Simplified Shampoo Update

- **Shampoo Update**:
  - The momentum matrix m is decomposed using SVD: $m = U\Sigma V^T$.
  - The update is then computed as UV^T, effectively orthogonalizing the update.
- **Why Orthogonalize?**:
  - Orthogonalizing the update helps in stabilizing the training process by preventing the gradients from becoming too large or too small in any particular direction.

Implementation of `SimpleShampoo`

Here's how you can implement the `SimpleShampoo` optimizer:

Copy

```
from torch.optim.optimizer import Optimizer
import torch

class SimpleShampoo(Optimizer):
    def __init__(self, params, lr=1e-1, b1=0.9):
        defaults = dict(lr=lr, b1=b1)
        super(SimpleShampoo, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group["params"]:
                if p.grad is None:
                    continue
                grad = p.grad.data
                state = self.state.setdefault(p, {})

                if len(state) == 0:
                    state["step"] = torch.tensor(0.0)
                    state["momentum"] = torch.zeros_like(p)
                state["step"] += 1
                m = state["momentum"]
                m.lerp_(grad, 1 - group["b1"])

                # Perform SVD on the momentum matrix
                if len(m.shape) > 1:  # Only for weight matrices, not biases
                    U, _, Vh = torch.linalg.svd(m, full_matrices=False)
                    update = U @ Vh
                else:
                    update = m  # For biases, no SVD

                p.add_(update, alpha=-group["lr"])
        return None
```

How to Use

```
model = MLP(hidden_sizes=[8, 16, 64, 64, 64, 256, 256, 1024]).to(device)
optimizer = SimpleShampoo(model.parameters(), lr=0.1)
train_one_step(optimizer=optimizer, label="Shampoo")
```

Expected Behavior

- **Stabilized Training**:
  - The Shampoo update should stabilize the training process by preventing extreme gradient magnitudes in any direction.
  - This can lead to more consistent convergence across different network architectures and learning rates.

Plotting the Norms

To compare the Frobenius and Spectral norms of the updates for Adam and Shampoo, you can modify the plotting code from part b) to include the Shampoo optimizer:

Explanation of the Code

- **Initialization**:
  - The optimizer initializes the momentum buffer for each parameter.
- **Momentum Update**:
  - The momentum is updated using an exponential moving average of the gradients.
- **SVD and Update**:
  - For weight matrices (2D tensors), the momentum matrix is decomposed using SVD: $m = U\Sigma V^T$.
  - The update is computed as UV^T, which is the orthogonalized version of the momentum matrix.
  - For biases (1D tensors), no SVD is performed, and the update is simply the momentum.
- **Parameter Update**:
  - The parameters are updated using the computed update matrix.

Plotting the Norms

To compare the Frobenius and Spectral norms of the updates for Adam and Shampoo, you can modify the plotting code from part b) to include the Shampoo optimizer:

```
def train_one_step_matrices(optimizer_class, label="Shampoo", lr=0.1):
    model = MLP(hidden_sizes=[8, 16, 64, 64, 64, 256, 256, 1024]).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer_class(model.parameters(), lr=lr)

    old_params = [p.detach().clone() for p in model.parameters()]

    for i in range(1):
        images_batch = train_images[batch_ids]
        labels_batch = train_labels[batch_ids]
        images_batch, labels_batch = images_batch.to(device), labels_batch.to(device)

        optimizer.zero_grad()
        outputs, activations = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

    new_params = [p.detach().clone() for p in model.parameters()]
```

```
    for p in delta_params:
        if len(p.shape) == 2:  # Only for weight matrices
            frob_norm = torch.linalg.norm(p, ord='fro')
            spectral_norm = torch.linalg.norm(p, ord=2)
            frob_norms.append(frob_norm.item())
            spectral_norms.append(spectral_norm.item())
            p_shapes.append(f"{p.shape[0]}x{p.shape[1]}")

    fig, axs = plt.subplots(1, 2, figsize=(12, 6))

    axs[0].set_title(f"Frobenius norm of update per layer ({label})")
    axs[0].set_xlabel("Hidden Size of layer")
    axs[0].bar(np.arange(len(frob_norms)), frob_norms)
    axs[0].set_xticks(np.arange(len(frob_norms)))
    axs[0].set_xticklabels(p_shapes, rotation=45, ha='right')

    axs[1].set_title(f"Spectral norm of update per layer ({label})")
    axs[1].set_xlabel("Hidden Size of layer")
    axs[1].bar(np.arange(len(spectral_norms)), spectral_norms)
    axs[1].set_xticks(np.arange(len(spectral_norms)))
    axs[1].set_xticklabels(p_shapes, rotation=45, ha='right')

    plt.show()

train_one_step_matrices(SimpleAdam, label="Adam", lr=0.01)
train_one_step_matrices(SimpleShampoo, label="Shampoo", lr=0.1)
```

Plotting the Norms

You can use the plotting code from part b) to compare the Frobenius and Spectral norms of the updates for Adam and Shampoo. Here's how you can call it:

```
train_one_step_matrices(optimizer=SimpleShampoo, label="Shampoo", lr=0.1)
```

Copy

Interpreting the Results

- **Frobenius Norm**:
  - The Frobenius norm measures the overall magnitude of the update matrix.
  - Shampoo should show more consistent Frobenius norms across layers compared to Adam.
- **Spectral Norm**:
  - The spectral norm measures the largest singular value of the update matrix.
  - Shampoo should show smaller and more consistent spectral norms, indicating more stable updates.

Interpreting the Results

- **Frobenius Norm**:
  - The Frobenius norm measures the overall magnitude of the update matrix.
  - Shampoo should show more consistent Frobenius norms across layers compared to Adam.
- **Spectral Norm**:
  - The spectral norm measures the largest singular value of the update matrix.
  - Shampoo should show smaller and more consistent spectral norms, indicating more stable updates.

Comment 5: After forcing it to stay up with the original version of the code, Le Chat was able to correct its behavior and arrive at the correct solution.